Cognitive Workload Allocation Under Uncertainty

A Stochastic Multiple Knapsack Approach with Workload Balance

Don Ma, Sujun Fang

April 2026

Agenda

Outline

Motivation: Cognitive Overload, Burnout

- Cognitive overload from excessive demands leads to billions of lost working days annually [Robinson, 2024].
- Productivity depends on **attention management** rather than rigid time scheduling [Grant, 2019].
- Traditional time management fails because it ignores the finite nature of human lifespan and cognitive capacity [Hart, 2024].
- Chronic illness, burnout risk, and work-family conflict underscore the need for systematic resource allocation that accounts for well-being [Cook and Zill, 2024, Yoo and Lee, 2022].

Outline

Problem Setting

A planner assigns a set of tasks $\mathcal{I}$ to a set of agents $\mathcal{J}$ over a finite planning horizon.

Problem Setting

A planner assigns a set of tasks $\mathcal{I}$ to a set of agents $\mathcal{J}$ over a finite planning horizon.

- Each task $i \in \mathcal{I}$ can be assigned to at most one agent $j \in \mathcal{J}$.

Problem Setting

A planner assigns a set of tasks $\mathcal{I}$ to a set of agents $\mathcal{J}$ over a finite planning horizon.

- Each task $i \in \mathcal{I}$ can be assigned to at most one agent $j \in \mathcal{J}$.
- Assignment of task i to agent j produces expected return R_{ij} and consumes random workload S_{ij}^{ω} under scenario ω.

Problem Setting

A planner assigns a set of tasks $\mathcal{I}$ to a set of agents $\mathcal{J}$ over a finite planning horizon.

- Each task $i \in \mathcal{I}$ can be assigned to at most one agent $j \in \mathcal{J}$.
- Assignment of task i to agent j produces expected return R_{ij} and consumes random workload S_{ij}^{ω} under scenario ω.
- Each agent j has limited workload capacity C_j.

Problem Setting

A planner assigns a set of tasks $\mathcal{I}$ to a set of agents $\mathcal{J}$ over a finite planning horizon.

- Each task $i \in \mathcal{I}$ can be assigned to at most one agent $j \in \mathcal{J}$.
- Assignment of task i to agent j produces expected return R_{ij} and consumes random workload S_{ij}^{ω} under scenario ω.
- Each agent j has limited workload capacity C_j.
- The planner allows a small overload probability $\alpha \in (0, 1)$ per agent (chance constraint).

Problem Setting

A planner assigns a set of tasks $\mathcal{I}$ to a set of agents $\mathcal{J}$ over a finite planning horizon.

- Each task $i \in \mathcal{I}$ can be assigned to at most one agent $j \in \mathcal{J}$.
- Assignment of task i to agent j produces expected return R_{ij} and consumes random workload S_{ij}^{ω} under scenario ω.
- Each agent j has limited workload capacity C_j.
- The planner allows a small overload probability $\alpha \in (0, 1)$ per agent (chance constraint).
- A workload-balance penalty $\lambda \geq 0$ discourages uneven utilization across agents.

Sets, Variables, and Parameters

Sets and indices

- Tasks $i \in \mathcal{I}$, Agents $j \in \mathcal{J}$

Sets, Variables, and Parameters

Sets and indices

- Tasks $i \in \mathcal{I}$, Agents $j \in \mathcal{J}$
- Scenarios $\omega \in \Omega$ with probabilities p_ω

Sets, Variables, and Parameters

Sets and indices

- Tasks $i \in \mathcal{I}$, Agents $j \in \mathcal{J}$
- Scenarios $\omega \in \Omega$ with probabilities p_ω

Decision variables

$$x_{ij} = \begin{cases} 1, & \text{task } i \to \text{agent } j \\ 0, & \text{otherwise} \end{cases}$$

Sets, Variables, and Parameters

Sets and indices

- Tasks $i \in \mathcal{I}$, Agents $j \in \mathcal{J}$
- Scenarios $\omega \in \Omega$ with probabilities p_ω

Decision variables

$$x_{ij} = \begin{cases} 1, & \text{task } i \to \text{agent } j \\ 0, & \text{otherwise} \end{cases}$$

Parameters

- R_{ij}: expected return
- S_{ij}^{ω}: workload in scenario ω
- C_j: agent capacity
- α: max overload probability
- λ: balance penalty weight

Sets, Variables, and Parameters

Sets and indices

- Tasks $i \in \mathcal{I}$, Agents $j \in \mathcal{J}$
- Scenarios $\omega \in \Omega$ with probabilities p_ω

Decision variables

$$x_{ij} = \begin{cases} 1, & \text{task } i \to \text{agent } j \\ 0, & \text{otherwise} \end{cases}$$

Parameters

- R_{ij}: expected return
- S_{ij}^{ω}: workload in scenario ω
- C_j: agent capacity
- α: max overload probability
- λ: balance penalty weight

Auxiliary variables

- $y_j^{\omega} \in \{0, 1\}$: violation indicator
- $U_j \geq 0$: expected utilization
- $Z \geq 0$: max utilization gap

Scenario-Based Chance Constraint

Workload uncertainty is represented by a finite set of scenarios Ω:

Scenario-Based Chance Constraint

Workload uncertainty is represented by a finite set of scenarios Ω:

- Each scenario ω has probability p_ω with $\sum_{\omega \in \Omega} p_\omega = 1$.

Scenario-Based Chance Constraint

Workload uncertainty is represented by a finite set of scenarios Ω:

- Each scenario ω has probability p_ω with $\sum_{\omega \in \Omega} p_\omega = 1$.
- In scenario ω, task i assigned to agent j requires workload S_{ij}^{ω}.

Scenario-Based Chance Constraint

Workload uncertainty is represented by a finite set of scenarios Ω:

- Each scenario ω has probability p_ω with $\sum_{\omega \in \Omega} p_\omega = 1$.
- In scenario ω, task i assigned to agent j requires workload S_{ij}^{ω}.

The violation indicator $y_j^{\omega} = 1$ marks scenario ω as an overload scenario for agent j:

$$\sum_{i \in \mathcal{I}} S_{ij}^{\omega} \, x_{ij} \;\leq\; C_j + M \, y_j^{\omega}, \quad \forall j \in \mathcal{J}, \; \forall \omega \in \Omega.$$

Scenario-Based Chance Constraint

Workload uncertainty is represented by a finite set of scenarios Ω:

- Each scenario ω has probability p_ω with $\sum_{\omega \in \Omega} p_\omega = 1$.
- In scenario ω, task i assigned to agent j requires workload S_{ij}^{ω}.

The violation indicator $y_j^{\omega} = 1$ marks scenario ω as an overload scenario for agent j:

$$\sum_{i \in \mathcal{I}} S_{ij}^{\omega} \, x_{ij} \;\leq\; C_j + M \, y_j^{\omega}, \quad \forall j \in \mathcal{J}, \; \forall \omega \in \Omega.$$

The total violation probability for each agent must stay below α:

$$\sum_{\omega \in \Omega} p_\omega \, y_j^{\omega} \;\leq\; \alpha, \quad \forall j \in \mathcal{J}.$$

Workload Utilization and Balance

Expected workload utilization of agent j, normalized by capacity:

$$U_j \;=\; \frac{1}{C_j} \sum_{i \in \mathcal{I}} \sum_{\omega \in \Omega} p_\omega \, S_{ij}^{\omega} \, x_{ij}, \quad \forall j \in \mathcal{J}.$$

Workload Utilization and Balance

Expected workload utilization of agent j, normalized by capacity:

$$U_j \;=\; \frac{1}{C_j} \sum_{i \in \mathcal{I}} \sum_{\omega \in \Omega} p_\omega \, S_{ij}^{\omega} \, x_{ij}, \quad \forall j \in \mathcal{J}.$$

Workload imbalance measure: Z bounds the largest pairwise utilization gap:

$$U_j - U_k \;\leq\; Z, \quad \forall j \in \mathcal{J},\ \forall k \in \mathcal{J}.$$

Workload Utilization and Balance

Expected workload utilization of agent j, normalized by capacity:

$$U_j \;=\; \frac{1}{C_j} \sum_{i \in \mathcal{I}} \sum_{\omega \in \Omega} p_\omega \, S_{ij}^{\omega} \, x_{ij}, \quad \forall j \in \mathcal{J}.$$

Workload imbalance measure: Z bounds the largest pairwise utilization gap:

$$U_j - U_k \;\leq\; Z, \quad \forall j \in \mathcal{J},\ \forall k \in \mathcal{J}.$$

- A smaller Z means a more balanced allocation.
- Normalization by C_j allows fair comparison across agents with different capacities.

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \qquad \text{(Obj)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_{j} \sum_{i} R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \qquad \text{(Assign)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(Assign)}$$

$$\sum_i S_{ij}^{\omega}\, x_{ij} \le C_j + M\, y_j^{\omega}, \quad \forall j,\ \forall \omega \in \Omega \quad \text{(Cap)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(Assign)}$$

$$\sum_i S_{ij}^{\omega}\, x_{ij} \le C_j + M\, y_j^{\omega}, \quad \forall j,\ \forall \omega \in \Omega \quad \text{(Cap)}$$

$$\sum_{\omega \in \Omega} p_\omega\, y_j^{\omega} \le \alpha, \quad \forall j \quad \text{(Chance)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(Assign)}$$

$$\sum_i S_{ij}^{\omega}\, x_{ij} \le C_j + M\, y_j^{\omega}, \quad \forall j,\ \forall \omega \in \Omega \quad \text{(Cap)}$$

$$\sum_{\omega \in \Omega} p_\omega\, y_j^{\omega} \le \alpha, \quad \forall j \quad \text{(Chance)}$$

$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \sum_{\omega \in \Omega} p_\omega S_{ij}^{\omega} x_{ij}, \quad \forall j \quad \text{(Util)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(Assign)}$$

$$\sum_i S_{ij}^{\omega}\, x_{ij} \le C_j + M\, y_j^{\omega}, \quad \forall j,\ \forall \omega \in \Omega \quad \text{(Cap)}$$

$$\sum_{\omega \in \Omega} p_\omega\, y_j^{\omega} \le \alpha, \quad \forall j \quad \text{(Chance)}$$

$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \sum_{\omega \in \Omega} p_\omega S_{ij}^{\omega} x_{ij}, \quad \forall j \quad \text{(Util)}$$

$$U_j - U_k \le Z, \quad \forall j, k \quad \text{(Bal)}$$

Full Formulation

$$\max_{x,y,U,Z} \quad \underbrace{\sum_j \sum_i R_{ij}\, x_{ij}}_{\text{total return}} \; - \; \underbrace{\lambda Z}_{\text{balance penalty}} \qquad \text{(Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(Assign)}$$

$$\sum_i S_{ij}^{\omega}\, x_{ij} \le C_j + M\, y_j^{\omega}, \quad \forall j,\ \forall \omega \in \Omega \quad \text{(Cap)}$$

$$\sum_{\omega \in \Omega} p_\omega\, y_j^{\omega} \le \alpha, \quad \forall j \quad \text{(Chance)}$$

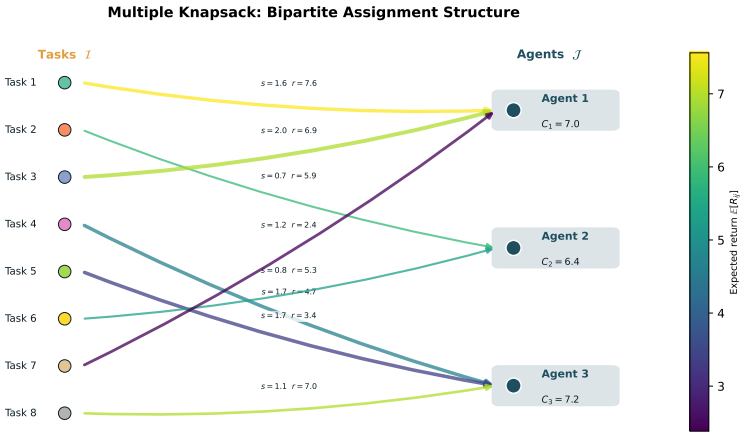
$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \sum_{\omega \in \Omega} p_\omega S_{ij}^{\omega} x_{ij}, \quad \forall j \quad \text{(Util)}$$

$$U_j - U_k \le Z, \quad \forall j, k \quad \text{(Bal)}$$

The objective balances three forces: return creation, overload-risk control, and workload fairness.

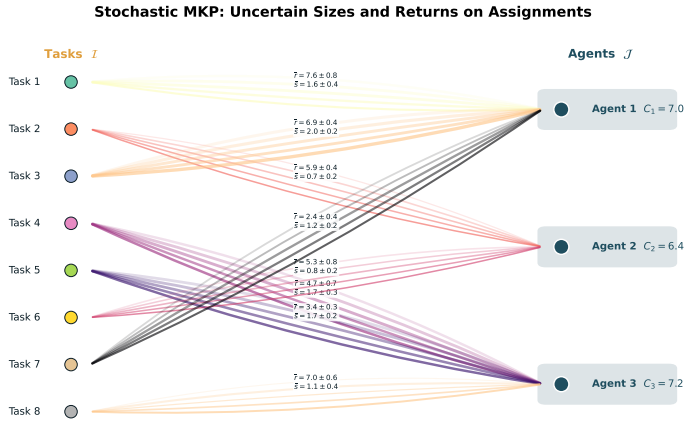
Outline

Multiple Knapsack Structure



Bipartite assignment: tasks (left) are assigned to agents (right) with capacities C_j. Edge width encodes task size $\mathbb{E}[S_{ij}]$ and color indicates expected return R_{ij}. Each task is assigned to at most one agent (constraint 10).

Stochastic Sizes and Returns



Stochastic extensions: each assignment edge shows multiple scenario realizations of size S_{ij}^{ω} (line width) and return R_{ij} (annotation). The chance constraint (10) limits the probability that total realized load exceeds agent capacity.

Outline

Stochastic Knapsack: Cutting Planes and Dominance

- Büyüktahtakın [2023] derive **scenario dominance cuts** for multi-stage stochastic MIPs applied to the dynamic multi-dimensional knapsack. Strong dominance cuts reduce solution time with 0.06% average deviation from optimality.
- Büyüktahtakın [2022] generalize this via **Stage-t Scenario Dominance** for risk-averse (mean-CVaR) knapsack problems, achieving one-to-two orders of magnitude speedup.
- Joung et al. [2023] compare LP relaxation bounds for the robust knapsack problem and show that extended formulations yield identical bounds.

Distributionally Robust and Risk-averse Knapsack

- Bos et al. [2024] approximate the stochastic knapsack with periodic scheduled arrivals using **Wasserstein Distributed Robust Optimization (DRO)**, producing robust healthcare surgical schedules.
- Ding et al. [2022] propose a target-based DRO knapsack with **uncertain profit and capacity**, reformulated as a second-order conic program.
- Merzifonluoglu and Geunes [2021] study the **risk-averse static stochastic knapsack** with CVaR and mean-standard deviation objectives, deriving efficient heuristics for normally distributed resource consumption.
- Boonstra et al. [2025] solve a robust multi-item newsvendor with budget constraint using a greedy knapsack algorithm under moment ambiguity.

Solution Algorithms for Stochastic Programs

- Chen and Luedtke [2022] analyze **sample average approximation** for two-stage programs without complete recourse, proving exponential convergence of recourse likelihood.
- Larsen et al. [2024] accelerate L-shaped methods via **supervised ML**, solving stochastic multi-knapsack instances in 20% of the time with less than 0.1% optimality gap.
- Lefebvre et al. [2024] reformulate adjustable robust optimization with **discrete uncertainty** and apply branch-and-cut to the multiple knapsack problem with uncertain weights.
- Lyu et al. [2022] unify responsive, adaptive, and anticipative allocation policies for the stochastic knapsack through **persistency values**.

Assignment, Scheduling, and Workforce Planning

- Rezaeian et al. [2024] model the **stochastic assignment of projects** to managers via multi-stage stochastic integer programming with heuristic algorithms for large instances.
- Nasirian et al. [2025] solve multiskilled workforce staffing and scheduling via **logic-based Benders decomposition**, achieving 29x speedup over direct MILP solving.
- Cavagnini et al. [2020] use two-stage stochastic programming for production planning under **uncertain learning rates**, with insights on cross-training and worker assignment.
- Novak et al. [2022] connect stochastic parallel machine scheduling to an **inflatable stochastic knapsack** pricing subproblem within branch-and-price.

Outline

Why Benders Decomposition?

- The full formulation carries one binary variable y_j^ω and one capacity constraint per agent-scenario pair.

Why Benders Decomposition?

- The full formulation carries one binary variable y_j^ω and one capacity constraint per agent-scenario pair.
- As the number of agents, tasks, and scenarios grows, solving the compact model directly becomes expensive.

Why Benders Decomposition?

- The full formulation carries one binary variable y_j^ω and one capacity constraint per agent-scenario pair.
- As the number of agents, tasks, and scenarios grows, solving the compact model directly becomes expensive.
- **Key insight**: the scenario-based chance constraints can be checked *after* a candidate assignment is fixed.

Why Benders Decomposition?

- The full formulation carries one binary variable y_j^ω and one capacity constraint per agent-scenario pair.
- As the number of agents, tasks, and scenarios grows, solving the compact model directly becomes expensive.
- **Key insight**: the scenario-based chance constraints can be checked *after* a candidate assignment is fixed.
- Benders decomposition separates the problem into:
  - A **master problem** that decides assignments and evaluates the return-balance trade-off.
  - A **feasibility subproblem** that checks whether the candidate assignment satisfies chance constraints.

Why Benders Decomposition?

- The full formulation carries one binary variable y_j^ω and one capacity constraint per agent-scenario pair.
- As the number of agents, tasks, and scenarios grows, solving the compact model directly becomes expensive.
- **Key insight**: the scenario-based chance constraints can be checked *after* a candidate assignment is fixed.
- Benders decomposition separates the problem into:
  - A **master problem** that decides assignments and evaluates the return-balance trade-off.
  - A **feasibility subproblem** that checks whether the candidate assignment satisfies chance constraints.
- This mirrors the managerial logic: propose an assignment first, then verify safety under uncertainty.

Benders Decomposition: Key Idea

Given a program with complicating variables $y \in Y$ and easy variables $x \geq 0$, Benders decomposition reformulates it as a master problem over y alone:

Benders Decomposition: Key Idea

Given a program with complicating variables $y \in Y$ and easy variables $x \geq 0$, Benders decomposition reformulates it as a master problem over y alone:

$$\min_{y \in Y} \; z + d'y$$

Benders Decomposition: Key Idea

Given a program with complicating variables $y \in Y$ and easy variables $x \geq 0$, Benders decomposition reformulates it as a master problem over y alone:

$$\min_{y \in Y} \; z + d'y$$

subject to **optimality cuts** (from dual extreme points π_p, $p \in \mathcal{P}$):

$$z \;\geq\; \pi_p'(e - By), \quad \forall p \in \mathcal{P}$$

Benders Decomposition: Key Idea

Given a program with complicating variables $y \in Y$ and easy variables $x \geq 0$, Benders decomposition reformulates it as a master problem over y alone:

$$\min_{y \in Y} \; z + d'y$$

subject to **optimality cuts** (from dual extreme points π_p, $p \in \mathcal{P}$):

$$z \;\geq\; \pi_p'(e - By), \quad \forall p \in \mathcal{P}$$

and **feasibility cuts** (from dual extreme rays ρ_r, $r \in \mathcal{R}$):

$$\rho_r'(e - By) \leq 0, \quad \forall r \in \mathcal{R}$$

Benders Decomposition: Key Idea

Given a program with complicating variables $y \in Y$ and easy variables $x \geq 0$, Benders decomposition reformulates it as a master problem over y alone:

$$\min_{y \in Y} \; z + d'y$$

subject to **optimality cuts** (from dual extreme points π_p, $p \in \mathcal{P}$):

$$z \;\geq\; \pi_p'(e - By), \quad \forall p \in \mathcal{P}$$

and **feasibility cuts** (from dual extreme rays ρ_r, $r \in \mathcal{R}$):

$$\rho_r'(e - By) \leq 0, \quad \forall r \in \mathcal{R}$$

Cuts are generated iteratively rather than enumerated. Convergence is finite because the dual polyhedron has finitely many extreme points and rays.

Our Decomposition Structure

Notation. Define the expected workload coefficient:

$$\bar{S}_{ij} = \sum_{\omega \in \Omega} p_\omega \, S_{ij}^{\omega}, \quad \forall i \in \mathcal{I}, \; \forall j \in \mathcal{J}.$$

Our Decomposition Structure

Notation. Define the expected workload coefficient:

$$\bar{S}_{ij} = \sum_{\omega \in \Omega} p_\omega \, S_{ij}^{\omega}, \quad \forall i \in \mathcal{I}, \; \forall j \in \mathcal{J}.$$

The utilization then simplifies to:

$$U_j = \frac{1}{C_j} \sum_{i \in \mathcal{I}} \bar{S}_{ij} \, x_{ij}, \quad \forall j \in \mathcal{J}.$$

Our Decomposition Structure

Notation. Define the expected workload coefficient:

$$\bar{S}_{ij} = \sum_{\omega \in \Omega} p_\omega \, S_{ij}^{\omega}, \quad \forall i \in \mathcal{I}, \, \forall j \in \mathcal{J}.$$

The utilization then simplifies to:

$$U_j = \frac{1}{C_j} \sum_{i \in \mathcal{I}} \bar{S}_{ij} \, x_{ij}, \quad \forall j \in \mathcal{J}.$$

- The **workload-balance component** depends only on expected workload, so it stays in the master problem.
- The **scenario-based chance feasibility** is naturally evaluated only after a candidate assignment is fixed.

Master Problem (Iteration t)

Let $\mathcal{B}^t$ denote the set of feasibility cuts generated so far.

$$\max_{x,U,Z} \quad \sum_j \sum_i R_{ij}\, x_{ij} \;-\; \lambda\, Z \quad \text{(MP-Obj)}$$

Master Problem (Iteration t)

Let $\mathcal{B}^t$ denote the set of feasibility cuts generated so far.

$$\max_{x,U,Z} \quad \sum_j \sum_i R_{ij}\, x_{ij} \;-\; \lambda\, Z \quad \text{(MP-Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(MP-1)}$$

Master Problem (Iteration t)

Let $\mathcal{B}^t$ denote the set of feasibility cuts generated so far.

$$\max_{x,U,Z} \quad \sum_j \sum_i R_{ij}\, x_{ij} \;-\; \lambda\, Z \quad \text{(MP-Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(MP-1)}$$

$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \bar{S}_{ij}\, x_{ij}, \quad \forall j \quad \text{(MP-2)}$$

Master Problem (Iteration t)

Let $\mathcal{B}^t$ denote the set of feasibility cuts generated so far.

$$\max_{x,U,Z} \quad \sum_j \sum_i R_{ij}\, x_{ij} \;-\; \lambda\, Z \quad \text{(MP-Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(MP-1)}$$

$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \bar{S}_{ij}\, x_{ij}, \quad \forall j \quad \text{(MP-2)}$$

$$U_j - U_k \le Z, \quad \forall j, k \quad \text{(MP-3)}$$

Master Problem (Iteration t)

Let $\mathcal{B}^t$ denote the set of feasibility cuts generated so far.

$$\max_{x,U,Z} \quad \sum_j \sum_i R_{ij}\, x_{ij} \;-\; \lambda\, Z \quad \text{(MP-Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(MP-1)}$$

$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \bar{S}_{ij}\, x_{ij}, \quad \forall j \quad \text{(MP-2)}$$

$$U_j - U_k \le Z, \quad \forall j, k \quad \text{(MP-3)}$$

$$x \in \mathcal{B}^t \quad \text{(MP-4)}$$

Master Problem (Iteration t)

Let $\mathcal{B}^t$ denote the set of feasibility cuts generated so far.

$$\max_{x,U,Z} \quad \sum_j \sum_i R_{ij}\, x_{ij} \;-\; \lambda\, Z \quad \text{(MP-Obj)}$$

$$\text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \forall i \quad \text{(MP-1)}$$

$$U_j = \tfrac{1}{C_j} \textstyle\sum_i \bar{S}_{ij}\, x_{ij}, \quad \forall j \quad \text{(MP-2)}$$

$$U_j - U_k \le Z, \quad \forall j, k \quad \text{(MP-3)}$$

$$x \in \mathcal{B}^t \quad \text{(MP-4)}$$

The master keeps the true objective and balance structure. Chance constraints are *not* included here; they are checked in the feasibility subproblem.

Feasibility Evaluation Subproblem

Given a candidate assignment $\bar{x}$ from the master:

- **Step 1.** Compute realized workload in each scenario:

$$L_j^{\omega}(\bar{x}) = \sum_{i \in \mathcal{I}} S_{ij}^{\omega} \, \bar{x}_{ij}, \quad \forall j, \ \forall \omega.$$

Feasibility Evaluation Subproblem

Given a candidate assignment $\bar{x}$ from the master:

- **Step 1.** Compute realized workload in each scenario:

$$L_j^{\omega}(\bar{x}) = \sum_{i \in \mathcal{I}} S_{ij}^{\omega} \, \bar{x}_{ij}, \quad \forall j, \ \forall \omega.$$

- **Step 2.** Determine scenario violations:

$$\delta_j^{\omega}(\bar{x}) = \begin{cases} 1, & \text{if } L_j^{\omega}(\bar{x}) > C_j, \\ 0, & \text{otherwise.} \end{cases}$$

Feasibility Evaluation Subproblem

Given a candidate assignment $\bar{x}$ from the master:

- **Step 1.** Compute realized workload in each scenario:

$$L_j^{\omega}(\bar{x}) = \sum_{i \in \mathcal{I}} S_{ij}^{\omega}\, \bar{x}_{ij}, \quad \forall j,\ \forall \omega.$$

- **Step 2.** Determine scenario violations:

$$\delta_j^{\omega}(\bar{x}) = \begin{cases} 1, & \text{if } L_j^{\omega}(\bar{x}) > C_j, \\ 0, & \text{otherwise.} \end{cases}$$

- **Step 3.** Compute violation probability:

$$P_j^{\text{viol}}(\bar{x}) = \sum_{\omega \in \Omega} p_{\omega}\, \delta_j^{\omega}(\bar{x}).$$

Feasibility Evaluation Subproblem

Given a candidate assignment $\bar{x}$ from the master:

- **Step 1.** Compute realized workload in each scenario:

$$L_j^{\omega}(\bar{x}) = \sum_{i \in \mathcal{I}} S_{ij}^{\omega}\, \bar{x}_{ij}, \quad \forall j,\ \forall \omega.$$

- **Step 2.** Determine scenario violations:

$$\delta_j^{\omega}(\bar{x}) = \begin{cases} 1, & \text{if } L_j^{\omega}(\bar{x}) > C_j, \\ 0, & \text{otherwise.} \end{cases}$$

- **Step 3.** Compute violation probability:

$$P_j^{\text{viol}}(\bar{x}) = \sum_{\omega \in \Omega} p_{\omega}\, \delta_j^{\omega}(\bar{x}).$$

- **Step 4.** Check: $\bar{x}$ is chance-feasible if and only if $P_j^{\text{viol}}(\bar{x}) \leq \alpha$ for all $j \in \mathcal{J}$.

Feasibility Evaluation Subproblem

Given a candidate assignment $\bar{x}$ from the master:

- **Step 1.** Compute realized workload in each scenario:

$$L_j^{\omega}(\bar{x}) = \sum_{i \in \mathcal{I}} S_{ij}^{\omega}\, \bar{x}_{ij}, \quad \forall j,\ \forall \omega.$$

- **Step 2.** Determine scenario violations:

$$\delta_j^{\omega}(\bar{x}) = \begin{cases} 1, & \text{if } L_j^{\omega}(\bar{x}) > C_j, \\ 0, & \text{otherwise.} \end{cases}$$

- **Step 3.** Compute violation probability:

$$P_j^{\text{viol}}(\bar{x}) = \sum_{\omega \in \Omega} p_{\omega}\, \delta_j^{\omega}(\bar{x}).$$

- **Step 4.** Check: $\bar{x}$ is chance-feasible if and only if $P_j^{\text{viol}}(\bar{x}) \leq \alpha$ for all $j \in \mathcal{J}$.

This is a **logical feasibility check**, not another optimization problem. Once $\bar{x}$ is fixed, all quantities are computed directly.

Feasibility Cut Generation: No-Good Cut

Suppose agent j violates the chance constraint under candidate $\bar{x}$.

- Let $\mathcal{I}_j(\bar{x}) = \{i \in \mathcal{I} : \bar{x}_{ij} = 1\}$ be the **full bundle** of tasks assigned to agent j.

Feasibility Cut Generation: No-Good Cut

Suppose agent j violates the chance constraint under candidate $\bar{x}$.

- Let $\mathcal{I}_j(\bar{x}) = \{i \in \mathcal{I} : \bar{x}_{ij} = 1\}$ be the **full bundle** of tasks assigned to agent j.
- The simplest valid cut forbids reassigning the exact same bundle:

$$\sum_{i \in \mathcal{I}_j(\bar{x})} x_{ij} \;\leq\; |\mathcal{I}_j(\bar{x})| - 1. \quad \text{(No-good cut)}$$

Feasibility Cut Generation: No-Good Cut

Suppose agent j violates the chance constraint under candidate $\bar{x}$.

- Let $\mathcal{I}_j(\bar{x}) = \{i \in \mathcal{I} : \bar{x}_{ij} = 1\}$ be the **full bundle** of tasks assigned to agent j.
- The simplest valid cut forbids reassigning the exact same bundle:

$$\sum_{i \in \mathcal{I}_j(\bar{x})} x_{ij} \;\leq\; |\mathcal{I}_j(\bar{x})| - 1. \quad \text{(No-good cut)}$$

- **Guarantee**: eliminates the current infeasible assignment pattern, so the algorithm never revisits it.

Feasibility Cut Generation: No-Good Cut

Suppose agent j violates the chance constraint under candidate $\bar{x}$.

- Let $\mathcal{I}_j(\bar{x}) = \{i \in \mathcal{I} : \bar{x}_{ij} = 1\}$ be the **full bundle** of tasks assigned to agent j.
- The simplest valid cut forbids reassigning the exact same bundle:

$$\sum_{i \in \mathcal{I}_j(\bar{x})} x_{ij} \;\leq\; |\mathcal{I}_j(\bar{x})| - 1. \quad \text{(No-good cut)}$$

- **Guarantee**: eliminates the current infeasible assignment pattern, so the algorithm never revisits it.
- **Limitation**: only excludes the *entire* bundle. Other bundles that share the same offending tasks remain feasible to the master, so convergence can be slow.

Feasibility Cut Generation: No-Good Cut

Suppose agent j violates the chance constraint under candidate $\bar{x}$.

- Let $\mathcal{I}_j(\bar{x}) = \{i \in \mathcal{I} : \bar{x}_{ij} = 1\}$ be the **full bundle** of tasks assigned to agent j.
- The simplest valid cut forbids reassigning the exact same bundle:

$$\sum_{i \in \mathcal{I}_j(\bar{x})} x_{ij} \;\leq\; |\mathcal{I}_j(\bar{x})| - 1. \quad \text{(No-good cut)}$$

- **Guarantee**: eliminates the current infeasible assignment pattern, so the algorithm never revisits it.
- **Limitation**: only excludes the *entire* bundle. Other bundles that share the same offending tasks remain feasible to the master, so convergence can be slow.

The no-good cut ensures finite convergence but may require many iterations because it eliminates only one assignment at a time.

Feasibility Cut Generation: Stronger Cut

The stronger cut targets the **root cause** of the violation.

- Within the full bundle $\mathcal{I}_j(\bar{x})$, identify a **minimal offending subset** $\mathcal{B}_j(\bar{x}) \subseteq \mathcal{I}_j(\bar{x})$ such that

$$\sum_{\omega \in \Omega:\ \sum_{i \in \mathcal{B}_j(\bar{x})} S_{ij}^{\omega} > C_j} p_\omega \ > \ \alpha.$$

Feasibility Cut Generation: Stronger Cut

The stronger cut targets the **root cause** of the violation.

- Within the full bundle $\mathcal{I}_j(\bar{x})$, identify a **minimal offending subset** $\mathcal{B}_j(\bar{x}) \subseteq \mathcal{I}_j(\bar{x})$ such that

$$\sum_{\omega \in \Omega:\, \sum_{i \in \mathcal{B}_j(\bar{x})} S_{ij}^{\omega} > C_j} p_\omega \;>\; \alpha.$$

- This subset alone already causes the chance violation. Add the cut:

$$\sum_{i \in \mathcal{B}_j(\bar{x})} x_{ij} \;\leq\; |\mathcal{B}_j(\bar{x})| - 1. \quad \text{(Stronger cut)}$$

Feasibility Cut Generation: Stronger Cut

The stronger cut targets the **root cause** of the violation.

- Within the full bundle $\mathcal{I}_j(\bar{x})$, identify a **minimal offending subset** $\mathcal{B}_j(\bar{x}) \subseteq \mathcal{I}_j(\bar{x})$ such that

$$\sum_{\omega \in \Omega:\ \sum_{i \in \mathcal{B}_j(\bar{x})} S_{ij}^{\omega} > C_j} p_\omega \ > \ \alpha.$$

- This subset alone already causes the chance violation. Add the cut:

$$\sum_{i \in \mathcal{B}_j(\bar{x})} x_{ij} \ \leq \ |\mathcal{B}_j(\bar{x})| - 1. \quad \text{(Stronger cut)}$$

- Because $\mathcal{B}_j \subseteq \mathcal{I}_j$ and $|\mathcal{B}_j| \leq |\mathcal{I}_j|$, the stronger cut **dominates** the no-good cut: it eliminates *every* assignment that contains the offending subset, not just the one specific bundle.

Feasibility Cut Generation: Stronger Cut

The stronger cut targets the **root cause** of the violation.

- Within the full bundle $\mathcal{I}_j(\bar{x})$, identify a **minimal offending subset** $\mathcal{B}_j(\bar{x}) \subseteq \mathcal{I}_j(\bar{x})$ such that

$$\sum_{\omega \in \Omega:\ \sum_{i \in \mathcal{B}_j(\bar{x})} S_{ij}^{\omega} > C_j} p_\omega \ > \ \alpha.$$

- This subset alone already causes the chance violation. Add the cut:

$$\sum_{i \in \mathcal{B}_j(\bar{x})} x_{ij} \ \leq \ |\mathcal{B}_j(\bar{x})| - 1. \quad \text{(Stronger cut)}$$

- Because $\mathcal{B}_j \subseteq \mathcal{I}_j$ and $|\mathcal{B}_j| \leq |\mathcal{I}_j|$, the stronger cut **dominates** the no-good cut: it eliminates *every* assignment that contains the offending subset, not just the one specific bundle.

Practical effect

Fewer iterations are needed because each stronger cut removes a larger region of infeasible assignments from the master problem.

No-Good vs. Stronger Cut: Comparison

Example. Agent j is assigned tasks $\{1, 2, 3\}$ and violates the chance constraint. Suppose tasks $\{1, 3\}$ alone already cause violation.

No-Good vs. Stronger Cut: Comparison

Example. Agent j is assigned tasks $\{1, 2, 3\}$ and violates the chance constraint. Suppose tasks $\{1, 3\}$ alone already cause violation.

- **No-good cut** forbids $\{1, 2, 3\}$ together: $x_{1j} + x_{2j} + x_{3j} \leq 2$.
  The master can still assign $\{1, 3\}$ to j, which is also infeasible, requiring another iteration.

No-Good vs. Stronger Cut: Comparison

Example. Agent j is assigned tasks $\{1, 2, 3\}$ and violates the chance constraint. Suppose tasks $\{1, 3\}$ alone already cause violation.

- **No-good cut** forbids $\{1, 2, 3\}$ together: $x_{1j} + x_{2j} + x_{3j} \leq 2$.
  The master can still assign $\{1, 3\}$ to j, which is also infeasible, requiring another iteration.
- **Stronger cut** forbids $\{1, 3\}$ together: $x_{1j} + x_{3j} \leq 1$.
  This also eliminates $\{1, 2, 3\}$, $\{1, 3, 4\}$, and any other bundle containing the offending pair.

No-Good vs. Stronger Cut: Comparison

Example. Agent j is assigned tasks $\{1, 2, 3\}$ and violates the chance constraint. Suppose tasks $\{1, 3\}$ alone already cause violation.

- **No-good cut** forbids $\{1, 2, 3\}$ together: $x_{1j} + x_{2j} + x_{3j} \leq 2$.
  The master can still assign $\{1, 3\}$ to j, which is also infeasible, requiring another iteration.
- **Stronger cut** forbids $\{1, 3\}$ together: $x_{1j} + x_{3j} \leq 1$.
  This also eliminates $\{1, 2, 3\}$, $\{1, 3, 4\}$, and any other bundle containing the offending pair.

Property	No-good	Stronger
Assignments eliminated	1	many
Subset size	$\lvert\mathcal{I}_j\rvert$ (full)	$\lvert\mathcal{B}_j\rvert \leq \lvert\mathcal{I}_j\rvert$
Convergence speed	slower	faster
Validity guarantee	yes	yes

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.
2. **Solve Master Problem.** Solve (23) to obtain candidate $\bar{x}$.

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.
2. **Solve Master Problem.** Solve (23) to obtain candidate $\bar{x}$.
3. **Feasibility Check.** For each agent j, compute $L_j^{\omega}(\bar{x})$ in every scenario and evaluate $P_j^{\text{viol}}(\bar{x})$.

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.
2. **Solve Master Problem.** Solve (23) to obtain candidate $\bar{x}$.
3. **Feasibility Check.** For each agent j, compute $L_j^{\omega}(\bar{x})$ in every scenario and evaluate $P_j^{\text{viol}}(\bar{x})$.
4. **If feasible** ($P_j^{\text{viol}} \leq \alpha$ for all j): $\bar{x}$ is optimal. **Stop.**

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.
2. **Solve Master Problem.** Solve (23) to obtain candidate $\bar{x}$.
3. **Feasibility Check.** For each agent j, compute $L_j^{\omega}(\bar{x})$ in every scenario and evaluate $P_j^{\text{viol}}(\bar{x})$.
4. **If feasible** ($P_j^{\text{viol}} \leq \alpha$ for all j): $\bar{x}$ is optimal. **Stop.**
5. **If infeasible**: for each violating agent j, generate a **stronger cut** (or a no-good cut if no smaller offending subset is found) and add it to $\mathcal{B}^{t+1}$.

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.
2. **Solve Master Problem.** Solve (23) to obtain candidate $\bar{x}$.
3. **Feasibility Check.** For each agent j, compute $L_j^{\omega}(\bar{x})$ in every scenario and evaluate $P_j^{\text{viol}}(\bar{x})$.
4. **If feasible** ($P_j^{\text{viol}} \leq \alpha$ for all j): $\bar{x}$ is optimal. **Stop.**
5. **If infeasible**: for each violating agent j, generate a **stronger cut** (or a no-good cut if no smaller offending subset is found) and add it to $\mathcal{B}^{t+1}$.
6. **Update.** Set $t \leftarrow t + 1$. Return to Step 2.

Algorithm: Step-by-Step

1. **Initialize.** Set $t = 0$, $\mathcal{B}^0 = \emptyset$.
2. **Solve Master Problem.** Solve (23) to obtain candidate $\bar{x}$.
3. **Feasibility Check.** For each agent j, compute $L_j^{\omega}(\bar{x})$ in every scenario and evaluate $P_j^{\text{viol}}(\bar{x})$.
4. **If feasible** ($P_j^{\text{viol}} \leq \alpha$ for all j): $\bar{x}$ is optimal. **Stop.**
5. **If infeasible**: for each violating agent j, generate a **stronger cut** (or a no-good cut if no smaller offending subset is found) and add it to $\mathcal{B}^{t+1}$.
6. **Update.** Set $t \leftarrow t + 1$. Return to Step 2.

Convergence

Both cut types guarantee finite termination: each cut eliminates at least one infeasible assignment pattern, and the total number of binary patterns is finite. Stronger cuts accelerate convergence by eliminating multiple patterns per iteration.

Why This Decomposition Works

- **Exact objective in master**: every master solution is evaluated with respect to the exact return-balance trade-off.

Why This Decomposition Works

- **Exact objective in master**: every master solution is evaluated with respect to the exact return-balance trade-off.
- **Scenario-free master**: all agent-scenario binary variables y_j^ω are removed from the main optimization.

Why This Decomposition Works

- **Exact objective in master**: every master solution is evaluated with respect to the exact return-balance trade-off.
- **Scenario-free master**: all agent-scenario binary variables y_j^ω are removed from the main optimization.
- **Simple subproblem**: the feasibility check is purely arithmetic (no LP or MIP to solve).

Why This Decomposition Works

- **Exact objective in master**: every master solution is evaluated with respect to the exact return-balance trade-off.
- **Scenario-free master**: all agent-scenario binary variables y_j^ω are removed from the main optimization.
- **Simple subproblem**: the feasibility check is purely arithmetic (no LP or MIP to solve).
- **Managerial interpretation**: the planner proposes an assignment, then checks whether it is sufficiently safe under workload uncertainty.

Outline

Test Instances

Three instances with increasing size:

Instance	Tasks	Agents	Scenarios	α	λ
Small	5	3	3	0.20	4.0
Medium	10	5	5	0.20	5.0
Large	25	10	10	0.10	6.0

Test Instances

Three instances with increasing size:

Instance	Tasks	Agents	Scenarios	α	λ
Small	5	3	3	0.20	4.0
Medium	10	5	5	0.20	5.0
Large	25	10	10	0.10	6.0

- Capacities vary slightly across agents (e.g., $[10, 10, 11]$ for the small instance).
- The large instance uses a tighter risk tolerance ($\alpha = 0.10$).

Computational Summary

Instance	Objective	Return	Z	Iterations	Cuts	Gap
Small	51.50	53	0.376	2	1	0.00
Medium	77.11	79	0.377	52	220	0.00
Large	254.34	257	0.444	443	1958	0.00

Computational Summary

Instance	Objective	Return	Z	Iterations	Cuts	Gap
Small	51.50	53	0.376	2	1	0.00
Medium	77.11	79	0.377	52	220	0.00
Large	254.34	257	0.444	443	1958	0.00

- All instances close the Benders gap to zero.
- The number of iterations and cuts increases substantially with instance size: from 2 iterations / 1 cut (small) to 443 iterations / 1958 cuts (large).

Small Instance: Assignment and Utilization

Assignment

Task	Agent
1	Agent 3
2	Agent 2
3	Agent 3
4	Agent 2
5	Agent 1

Small Instance: Assignment and Utilization

Assignment

Task	Agent
1	Agent 3
2	Agent 2
3	Agent 3
4	Agent 2
5	Agent 1

Expected utilization

Agent	U_j
1	0.570
2	0.940
3	0.946

Small Instance: Assignment and Utilization

Assignment

Task	Agent
1	Agent 3
2	Agent 2
3	Agent 3
4	Agent 2
5	Agent 1

Expected utilization

Agent	U_j
1	0.570
2	0.940
3	0.946

- Solved in 2 iterations with 1 feasibility cut.
- The first master solution was infeasible because agent 1 received an unsafe bundle. One cut corrected this.

Medium Instance: Assignment and Utilization

Assignment

Task	Agent
2	Agent 2
3	Agent 3
4	Agent 4
5	Agent 5
7	Agent 1
10	Agent 4

Tasks 1, 6, 8, 9: **not assigned**.

Medium Instance: Assignment and Utilization

Assignment

Task	Agent
2	Agent 2
3	Agent 3
4	Agent 4
5	Agent 5
7	Agent 1
10	Agent 4

Tasks 1, 6, 8, 9: **not assigned**.

Expected utilization

Agent	U_j
1	0.545
2	0.628
3	0.545
4	0.922
5	0.645

Medium Instance: Assignment and Utilization

Assignment

Task	Agent
2	Agent 2
3	Agent 3
4	Agent 4
5	Agent 5
7	Agent 1
10	Agent 4

Expected utilization

Agent	U_j
1	0.545
2	0.628
3	0.545
4	0.922
5	0.645

Tasks 1, 6, 8, 9: **not assigned**.

- 52 iterations and 220 feasibility cuts.
- Leaving some tasks unassigned is optimal: assigning them would violate chance constraints or create too much imbalance.

Large Instance: Assignment and Utilization

Assignment highlights

- 19 of 25 tasks are assigned.
- Tasks 5, 11, 12, 17, 23, 24: not assigned.
- Most agents receive 2 tasks; agent 7 receives only 1.

Large Instance: Assignment and Utilization

Assignment highlights

- 19 of 25 tasks are assigned.
- Tasks 5, 11, 12, 17, 23, 24: not assigned.
- Most agents receive 2 tasks; agent 7 receives only 1.

Expected utilization

Agent	U_j
1	0.786
2	0.722
3	0.730
4	0.722
5	0.739
6	0.747
7	0.355
8	0.748
9	0.785
10	0.755

Large Instance: Assignment and Utilization

Assignment highlights

- 19 of 25 tasks are assigned.
- Tasks 5, 11, 12, 17, 23, 24: not assigned.
- Most agents receive 2 tasks; agent 7 receives only 1.

Expected utilization

Agent	U_j
1	0.786
2	0.722
3	0.730
4	0.722
5	0.739
6	0.747
7	0.355
8	0.748
9	0.785
10	0.755

443 iterations and 1958 cuts. Feasible but not perfectly balanced. Agent 7 is the lightest outlier.

Observations from Results

- **Scalability**: iterations and cuts grow substantially with instance size (from 2 to 443 iterations).

Observations from Results

- **Scalability**: iterations and cuts grow substantially with instance size (from 2 to 443 iterations).
- **Not all tasks are assigned**: the optimizer leaves tasks undone when assigning them would violate chance constraints or worsen imbalance.

Observations from Results

- **Scalability**: iterations and cuts grow substantially with instance size (from 2 to 443 iterations).
- **Not all tasks are assigned**: the optimizer leaves tasks undone when assigning them would violate chance constraints or worsen imbalance.
- **Workload balance**: the balance penalty λZ keeps utilization spread moderate, though perfect balance is not always achievable.

Observations from Results

- **Scalability**: iterations and cuts grow substantially with instance size (from 2 to 443 iterations).
- **Not all tasks are assigned**: the optimizer leaves tasks undone when assigning them would violate chance constraints or worsen imbalance.
- **Workload balance**: the balance penalty λZ keeps utilization spread moderate, though perfect balance is not always achievable.
- **Zero Benders gap**: all instances are solved to proven optimality, confirming correctness of the decomposition.

Observations from Results

- **Scalability**: iterations and cuts grow substantially with instance size (from 2 to 443 iterations).
- **Not all tasks are assigned**: the optimizer leaves tasks undone when assigning them would violate chance constraints or worsen imbalance.
- **Workload balance**: the balance penalty λZ keeps utilization spread moderate, though perfect balance is not always achievable.
- **Zero Benders gap**: all instances are solved to proven optimality, confirming correctness of the decomposition.
- **Computational cost**: the explicit outer-loop Benders scheme becomes expensive for large instances with tight chance constraints, suggesting room for acceleration strategies.

Outline

Key Takeaways

- The formulation models cognitive resource allocation as a **stochastic multiple knapsack** with scenario-based chance constraints and a workload-balance penalty.

Key Takeaways

- The formulation models cognitive resource allocation as a **stochastic multiple knapsack** with scenario-based chance constraints and a workload-balance penalty.
- **Benders decomposition** separates assignment decisions from chance-feasibility checks, keeping the exact objective in the master and using a simple logical subproblem.

Key Takeaways

- The formulation models cognitive resource allocation as a **stochastic multiple knapsack** with scenario-based chance constraints and a workload-balance penalty.
- **Benders decomposition** separates assignment decisions from chance-feasibility checks, keeping the exact objective in the master and using a simple logical subproblem.
- Computational results on three instances demonstrate correctness (zero gap) and reveal the scalability challenge as instance size grows.

Key Takeaways

- The formulation models cognitive resource allocation as a **stochastic multiple knapsack** with scenario-based chance constraints and a workload-balance penalty.
- **Benders decomposition** separates assignment decisions from chance-feasibility checks, keeping the exact objective in the master and using a simple logical subproblem.
- Computational results on three instances demonstrate correctness (zero gap) and reveal the scalability challenge as instance size grows.
- Future directions include stronger cut generation, scenario dominance [Büyüktahtakın, 2023], and ML-accelerated decomposition [Larsen et al., 2024].

Outline

References I

Guus Boonstra, Wouter J.E.C. van Eekelen, and Johan S.H. van Leeuwaarden. Robust Knapsack Ordering for A Partially-informed Newsvendor with Budget Constraint. *Operations Research Letters*, 58, 2025. doi: 10.1016/j.orl.2024.107202.

Hayo Bos, Richard J. Boucherie, Erwin W. Hans, and Gréanne Leeftink. Distributionally Robust Scheduling of Stochastic Knapsack Arrivals. *Computers and Operations Research*, 167, 2024. doi: 10.1016/j.cor.2024.106641.

Esra Büyüktahtakın. Scenario-dominance to Multi-stage Stochastic Lot-sizing and Knapsack Problems. *Computers and Operations Research*, 153, 2023. doi: 10.1016/j.cor.2023.106149.

I. Esra Büyüktahtakın. Stage-t Scenario Dominance for Risk-averse Multi-stage Stochastic Mixed-integer Programs. *Annals of Operations Research*, 309, 2022. doi: 10.1007/s10479-021-04388-3.

Rossana Cavagnini, Mike Hewitt, and Francesca Maggioni. Workforce Production Planning Under Uncertain Learning Rates. *International Journal of Production Economics*, 225, 2020. doi: 10.1016/j.ijpe.2019.107590.

References II

Rui Chen and James Luedtke. On Sample Average Approximation for Two-stage Stochastic Programs Without Relatively Complete Recourse. *Mathematical Programming*, 196: 719–754, 2022. doi: 10.1007/s10107-021-01753-9.

Alexandra Cook and Alexander Zill. Individual Health Status As A Resource: Analyzing Associations Between Perceived Illness Symptom Severity, Burnout, and Work Engagement Among Employees with Autoimmune Diseases. *Applied Psychology*, 73:990–1025, 2024. doi: 10.1111/apps.12464.

Jianpeng Ding, Liuxin Chen, Ginger Y. Ke, Yuanbo Li, and Lianmin Zhang. Balancing The Profit and Capacity Under Uncertainties: A Target-based Distributionally Robust Knapsack Problem. *International Transactions in Operational Research*, 29:760–782, 2022. doi: 10.1111/itor.13031.

References III

Adam Grant. Productivity isn't about time management. it's about attention management., March 2019. URL https://www.nytimes.com/2019/03/28/smarter-living/productivity-isnt-about-time-management-its-about-attention-management.html. Accessed: January 25, 2026.

Hanna Hart. Productivity and the hard truth about time management, January 2024. URL https://www.forbes.com/sites/hannahart/2024/01/25/productivity-and-the-hard-truth-about-time-management/. Accessed: January 25, 2026.

Seulgi Joung, Seyoung Oh, and Kyungsik Lee. Comparative Analysis of Linear Programming Relaxations for The Robust Knapsack Problem. *Annals of Operations Research*, 323:65–78, 2023. doi: 10.1007/s10479-022-05161-w.

Eric Larsen, Emma Frejinger, Bernard Gendron, and Andrea Lodi. Fast Continuous and Integer L-shaped Heuristics Through Supervised Learning. *Informs Journal on Computing*, 36: 203–223, 2024. doi: 10.1287/ijoc.2022.0175.

References IV

Henri Lefebvre, Enrico Malaguti, and Michele Monaci. Adjustable Robust Optimization with Discrete Uncertainty. *Informs Journal on Computing*, 36:78–96, 2024. doi: 10.1287/ijoc.2022.0086.

Guodong Lyu, Mabel C. Chou, Chung Piaw Teo, Zhichao Zheng, and Yuanguang Zhong. Stochastic Knapsack Revisited: The Service Level Perspective. *Operations Research*, 70: 729–747, 2022. doi: 10.1287/opre.2021.2173.

Yasemin Merzifonluoglu and Joseph Geunes. The Risk-averse Static Stochastic Knapsack Problem. *Informs Journal on Computing*, 33:931–948, 2021. doi: 10.1287/ijoc.2020.0972.

Araz Nasirian, Lele Zhang, Alysson M. Costa, and Babak Abbasi. Multiskilled Workforce Staffing and Scheduling: A Logic-based Benders' Decomposition Approach. *European Journal of Operational Research*, 323:20–33, 2025. doi: 10.1016/j.ejor.2024.11.033.

References V

Antonin Novak, Premysl Sucha, Matej Novotny, Richard Stec, and Zdenek Hanzalek. Scheduling Jobs with Normally Distributed Processing Times On Parallel Machines. *European Journal of Operational Research*, 297:422–441, 2022. doi: 10.1016/j.ejor.2021.05.011.

Aidin Rezaeian, Hamidreza Koosha, Mohammad Ranjbar, and Saeed Poormoaied. The Assignment of Project Managers to Projects in An Uncertain Dynamic Environment. *Annals of Operations Research*, 341:1107–1134, 2024. doi: 10.1007/s10479-024-05958-x.

Bryan Robinson. 4 steps to mitigate cognitive overload and career stress before 2025, November 2024. URL `https://www.forbes.com/sites/bryanrobinson/2024/11/02/4-steps-to-mitigate-cognitive-overload-and-career-stress-before-2025/`. Accessed: January 25, 2026.

Hye Jin Yoo and Hyeongsuk Lee. Critical Role of Information and Communication Technology in Nursing During The COVID-19 Pandemic: A Qualitative Study. *Journal of Nursing Management*, 30:3677–3685, 2022. doi: 10.1111/jonm.13880.